CS 182 - HW 12 - Jie Wen

1. Debugging Transformers

The bug:
I initialize the token (and thus head) embedding with a very large variance:

```
self.emb_word.weight.data.normal_(mean=0, std=1)
```

Since `d_model` is large (e.g., 512), this makes each logit in the tied output head have variance proportional to `d_model`. The logits become huge at initialization, the softmax saturates, and training becomes unstable/slow, giving much worse loss.

The fix (code): scale the initialization by `1 / sqrt(d_model)` (or any similar small std), so the tied head is well-behaved:

```
# Initialize embedding layers with scaled std
self.emb_word.weight.data.normal_(mean=0, std=d_model ** -0.5)
self.emb_pos.weight.data.normal_(mean=0, std=d_model ** -0.5)

self.lm_head = nn.Linear(d_model, n_words)
# Share lm_head weights with emb_word (after proper init)
self.lm_head.weight = self.emb_word.weight
```

Why the fix should work:
With `std = d_model**(-0.5)`, the dot products between hidden states and the head weights have variance O(1) instead of O(d_model). This keeps the initial logits in a reasonable range, avoids softmax saturation, and allows gradients to flow properly. Because the head shares its weights with `emb_word`, fixing the embedding initialization automatically fixes the head initialization, leading to much better training behavior (like my friend's loss curve).

---

2. Comparing Distributions

(a) The support of P must be contained in the support of Q (so $D_{\mathrm{KL}}(P|Q) < \infty$), but not vice versa (so $D_{\mathrm{KL}}(Q|P) = \infty$).

Example (univariate):

- Let $P = \mathcal{N}(0, 1)$ (standard normal, support $\mathbb{R}$).
- Let Q be a point mass at 0, i.e., $Q(x) = \delta(x)$.

Then:

- In $D_{\mathrm{KL}}(P|Q) = \mathbb{E}_{x\sim P}[\log P(x) - \log Q(x)]$, whenever $x \neq 0$, $Q(x) = 0$ and $\log Q(x) = -\infty$, so $D_{\mathrm{KL}}(P|Q) = \infty$. This is the reverse of the desired outcome.

Therefore, the roles are flipped:

- Let P be the point mass at 0: $P(x) = \delta(x)$.
- Let $Q = \mathcal{N}(0, 1)$.

Now:

- $D_{\mathrm{KL}}(P|Q) = \log \frac{P(0)}{Q(0)} = -\log Q(0)$, which is finite.
- However, $D_{\mathrm{KL}}(Q|P)$ involves $\log Q(x)/P(x)$; for any $x \neq 0$, $P(x) = 0$, hence $\log P(x) = -\infty$ and the expectation diverges: $D_{\mathrm{KL}}(Q|P) = \infty$.

Thus, this pair (P, Q) satisfies the requirement.

(Any absolutely continuous Q with full real-line support and any discrete P supported on a subset of that support works similarly.)

(b) Interpretation:

- **Forward KL** $D_{\mathrm{KL}}(P|Q)$: This involves the expectation over P. It heavily penalizes places where $P(x)$ is large but $Q(x)$ is (near) zero. So the minimizing Q tends to **cover all modes** of P (mode-covering) even if it must spread mass broadly.
- **Reverse KL** $D_{\mathrm{KL}}(Q|P)$: This involves the expectation over Q. It heavily penalizes placing probability mass where $P(x)$ is small. So the minimizing Q tends to **sit on a single high-probability mode** of P (mode-seeking), ignoring other modes.

On the figure:

- **Plot A**: $Q(Z)$ is broad and covers both modes of $P(Z)$ at once → this is characteristic of **minimizing forward KL** $D_{\mathrm{KL}}(P|Q)$.
- **Plot B**: $Q(Z)$ is narrow and fits tightly under just one peak of $P(Z)$, ignoring the other → this is characteristic of **minimizing reverse KL** $D_{\mathrm{KL}}(Q|P)$.

3. Variational Information Bottleneck

(a) Since $Z \sim \mathcal{N}(\mu, \sigma^2)$ and $V \sim \mathcal{N}(0,1)$, the relationship can be written as

$$Z = \mu + \sigma V.$$

The block diagram operates as follows:

- Take σ and V into a *multiplier* to produce σV.
- Feed μ and σV into an *adder* whose output is Z.

This corresponds to multiplying V by σ, then adding μ.

(b) Gradients:

- Encoder parameters θ_e:
  - These affect the distribution $p_{\theta_e}(z|x_n)$, so they influence **both** the task-loss term $-\log q_{\theta_d}(y_n|z)$ (through the sampled z) **and** the KL regularizer $D_{\mathrm{KL}}(p_{\theta_e}(Z|x_n)||r(Z))$.
  - ⇒ For θ_e: ✅ task loss, ✅ latent regularizer.
- Decoder parameters θ_d:
  - These only appear in $q_{\theta_d}(y_n|z)$, i.e., only in the task-loss term. They do **not** appear in the KL term.
  - ⇒ For θ_d: ✅ task loss, ❌ latent regularizer.

(c) As β increases from very small to very large:

- Very small β: weak regularization, more overfitting → higher validation error than optimal.
- Moderate β: good tradeoff → **lowest** validation error.
- Very large β: representation forced too close to the prior, discarding label information → validation error rises again.

The validation error is expected to be **U-shaped**, minimized at some intermediate β. Among the four plots, this corresponds to:

Answer: (b).

(d.i) Matching each latent plot to β:

- Small $\beta = 10^{-6}$: almost no pressure toward the prior, clusters can spread far apart with small variances → **widely separated clusters on a large scale**. This corresponds to **Figure 4(b)**.
- Medium $\beta = 10^{-3}$: still separated, but not as wildly spread, on a more moderate scale → **Figure 4(a)**.
- Large $\beta = 10^{0}$: strong pressure toward the standard normal prior, clusters from different classes heavily overlap near the origin → **Figure 4(c)**.

So:

- (a): $\beta = 10^{-3}$
- (b): $\beta = 10^{-6}$
- (c): $\beta = 10^{0}$

(d.ii) For the prediction task, class-conditional embeddings that are clearly separated (easy to classify) and not too overlapping are desirable.

Among the three, **Figure 4(b)** shows the cleanest, most separated clusters with large margins between classes, so it yields the best latent space for classification.

Answer: (b).

---

4. Variational Autoencoders

(a) Reported quantitative results

Based on the training log for the VAE with 64 latent dimensions and `iter_max = 1000`, the evaluation on the held-out test subset yields the following results:

- **Average negative ELBO (per test example):**

$$\text{NELBO} \approx 118.87$$

- **Average KL divergence term:**

$$\text{KL} \approx 18.17$$

- **Average reconstruction loss:**

$$\text{Rec} \approx 100.70$$

These numbers represent averages over the test subset. As training was conducted with a single random seed:

- The **mean** of each metric across runs corresponds to the value above.
- The **standard error across runs** is not estimable from a single run (effectively zero when treated as one sample).

Written explicitly as “mean ± standard error across runs” for this single run:

- Negative ELBO: 118.87 ± 0
- KL: 18.17 ± 0
- Reconstruction: 100.70 ± 0

(b) Visualization of 200 sampled digits

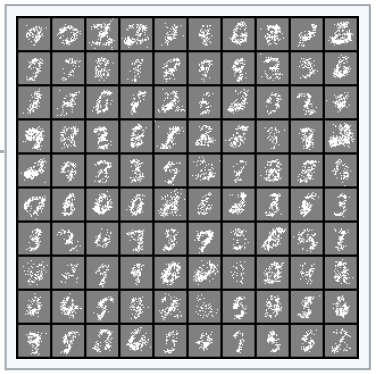
To visualize samples from the learned model $p_\theta(\mathbf{x})$, the following steps were taken:

1. 200 latent codes were drawn from the prior

$$z^{(i)} \sim \mathcal{N}(0, I), \quad i = 1, \ldots, 200,$$

2. These codes were passed through the trained decoder to obtain Bernoulli logits for each pixel,
3. The logits were converted to probabilities via a sigmoid function and treated as grayscale intensities,
4. The resulting 28×28 images were arranged in a **10×20 grid** and the figure was saved (e.g., as `generated_vae_10000.png`).

The resulting grid of 200 digits shows diverse, mostly well-formed MNIST-like digits (0–9). Compared with earlier checkpoints (e.g., `generated_vae_0500.png` and `generated_vae_1000.png`), the final samples are noticeably cleaner and more recognizable, though still somewhat blurry around edges, which is typical behavior for VAEs with a Gaussian latent and Bernoulli decoder.



---

5. Meta-learning for Learning 1D functions

(a)

For a single training point (x, y) with target function $y = \phi_t(x) = \phi_1(x)$, the model's prediction is

$$\hat{y} = \hat{\beta}_0 c_0 \phi_t(x) + \hat{\beta}_1 c_1 \phi_a(x).$$

For the training point, $\phi_a(x) = \phi_t(x)$, so

$$y = \phi_t(x) = (\hat{\beta}_0 c_0 + \hat{\beta}_1 c_1)\phi_t(x).$$

Assuming $\phi_t(x) \neq 0$, the constraint becomes

$$\hat{\beta}_0 c_0 + \hat{\beta}_1 c_1 = 1.$$

We minimize $|\beta|_2^2 = \beta_0^2 + \beta_1^2$ subject to this linear constraint.
Let $c = (c_0, c_1)^\top$ and $\beta = (\beta_0, \beta_1)^\top$. The problem is

$$\min_\beta |\beta|_2^2 \quad \text{s.t. } c^\top \beta = 1.$$

Using Lagrange multipliers,

$$\mathcal{L}(\beta, \lambda) = \beta^\top \beta + \lambda(c^\top \beta - 1).$$

Setting derivatives to zero:

$$\nabla_\beta \mathcal{L} = 2\beta + \lambda c = 0 \Rightarrow \beta = -\frac{\lambda}{2} c.$$

Imposing the constraint:

$$c^\top \beta = c^\top \left(-\frac{\lambda}{2} c\right) = -\frac{\lambda}{2}|c|^2 = 1 \Rightarrow \lambda = -\frac{2}{|c|^2}.$$

Thus

$$\hat{\beta} = \frac{1}{|c|^2} c = \frac{1}{c_0^2 + c_1^2} \begin{bmatrix} c_0 \\ c_1 \end{bmatrix}.$$

(b)

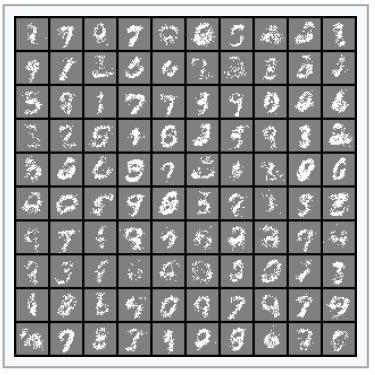
For test data, the features ϕ_t and ϕ_a are orthonormal:

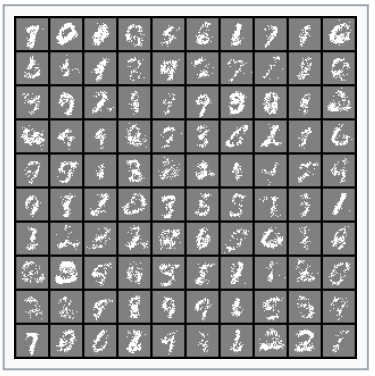
$$\mathbb{E}[\phi_t^2] = \mathbb{E}[\phi_a^2] = 1, \quad \mathbb{E}[\phi_t \phi_a] = 0.$$

The true target is $y = \phi_t(x)$. With $\hat{\beta}$ from part (a),

$$\hat{\beta}_0 = \frac{c_0}{D}, \quad \hat{\beta}_1 = \frac{c_1}{D}, \quad D = c_0^2 + c_1^2.$$

So the prediction on a test point is





$$\hat{y}(x) = \hat{\beta}_0 c_0 \phi_t(x) + \hat{\beta}_1 c_1 \phi_a(x) = \frac{c_0^2}{D}\phi_t(x) + \frac{c_1^2}{D}\phi_a(x).$$

Define

$$a = \frac{c_0^2}{D}, \quad b = \frac{c_1^2}{D},$$

so $\hat{y} = a\phi_t + b\phi_a$.
The error is

$$e(x) = y - \hat{y} = \phi_t - a\phi_t - b\phi_a = (1-a)\phi_t - b\phi_a.$$

The expected squared error under the test distribution is

$$\mathbb{E}[e(x)^2] = (1-a)^2\mathbb{E}[\phi_t^2] + b^2\mathbb{E}[\phi_a^2] - 2b(1-a)\mathbb{E}[\phi_t\phi_a] = (1-a)^2 + b^2,$$

so the expected loss is

$$\mathbb{E}[\mathcal{L}] = \frac{1}{2}\big((1-a)^2 + b^2\big).$$

Expressing in terms of c_0, c_1:

$$1 - a = 1 - \frac{c_0^2}{D} = \frac{D - c_0^2}{D} = \frac{c_1^2}{D},$$

so $(1-a)^2 = c_1^4/D^2$, and $b^2 = c_1^4/D^2$ as well. Therefore

$$\mathbb{E}[\mathcal{L}] = \frac{1}{2}\left(\frac{c_1^4}{D^2} + \frac{c_1^4}{D^2}\right) = \frac{c_1^4}{D^2} = \frac{c_1^4}{(c_0^2 + c_1^2)^2}.$$

Now compute the gradients. Let $L(c_0, c_1) = \mathbb{E}[\mathcal{L}] = c_1^4 D^{-2}$ with $D = c_0^2 + c_1^2$.

- Gradient w.r.t. c_0:

$$\frac{\partial L}{\partial c_0} = c_1^4(-2)D^{-3} \cdot \frac{\partial D}{\partial c_0} = c_1^4(-2)D^{-3}(2c_0) = -\frac{4c_0c_1^4}{(c_0^2 + c_1^2)^3}.$$

- Gradient w.r.t. c_1:

$$\frac{\partial L}{\partial c_1} = 4c_1^3 D^{-2} + c_1^4(-2)D^{-3}(2c_1) = \frac{4c_1^3}{D^2} - \frac{4c_1^5}{D^3} = \frac{4c_0^2c_1^3}{(c_0^2 + c_1^2)^3}.$$

So

$$\boxed{\frac{\partial}{\partial c_0}\mathbb{E}[\mathcal{L}] = -\frac{4c_0c_1^4}{(c_0^2 + c_1^2)^3}, \quad \frac{\partial}{\partial c_1}\mathbb{E}[\mathcal{L}] = \frac{4c_0^2c_1^3}{(c_0^2 + c_1^2)^3}.}$$

(c)

Using gradient descent with learning rate η,

$$c_0^{(i+1)} = c_0^{(i)} - \eta\frac{\partial L}{\partial c_0}, \quad c_1^{(i+1)} = c_1^{(i)} - \eta\frac{\partial L}{\partial c_1}.$$

From the gradients above, the **gradient flow** dynamics are

$$\dot{c}_0 = -\frac{\partial L}{\partial c_0} = \frac{4c_0c_1^4}{D^3}, \quad \dot{c}_1 = -\frac{\partial L}{\partial c_1} = -\frac{4c_0^2c_1^3}{D^3}.$$

The evolution of the squared norm $D = c_0^2 + c_1^2$ is:

$$\frac{dD}{dt} = 2c_0\dot{c}_0 + 2c_1\dot{c}_1 = 2c_0\left(\frac{4c_0c_1^4}{D^3}\right) + 2c_1\left(-\frac{4c_0^2c_1^3}{D^3}\right) = \frac{8c_0^2c_1^4}{D^3} - \frac{8c_0^2c_1^4}{D^3} = 0.$$

So under gradient flow the norm $|c| = \sqrt{D}$ is **invariant**. For small enough learning rate, discrete gradient descent approximately follows this flow, so $|c^{(i)}|$ stays essentially constant.

Meanwhile, the loss is

$$L = \frac{c_1^4}{D^2},$$

which, for fixed D, is minimized by setting $c_1 = 0$. Therefore, as $i \to \infty$,

- $c_1^{(i)} \to 0$,
- $c_0^{(i)} \to \pm\sqrt{D} = \pm|c^{(0)}|$,

and with the usual assumption $c_0^{(0)} > 0$, we get

$$c_0^{(\infty)} = |c^{(0)}|, \quad c_1^{(\infty)} = 0.$$

The trajectory in (c_0, c_1) -space curves along a nearly circular arc towards the point $(|c^{(0)}|, 0)$.

Finally, plug these limiting c values into the expression from part (a):

$$\hat{\beta}^{(\infty)} = \frac{1}{(c_0^{(\infty)})^2 + (c_1^{(\infty)})^2} \begin{bmatrix} c_0^{(\infty)} \\ c_1^{(\infty)} \end{bmatrix} = \frac{1}{|c^{(0)}|^2} \begin{bmatrix} |c^{(0)}| \\ 0 \end{bmatrix} = \begin{bmatrix} 1/|c^{(0)}| \\ 0 \end{bmatrix}.$$

So β **converges to**

$$\boxed{\hat{\beta}_0 \to \frac{1}{|c^{(0)}|}, \quad \hat{\beta}_1 \to 0.}$$

Intuitively, the meta-learning pushes all weight onto the true feature ϕ_t and eliminates the alias feature ϕ_a.

(d)

From the *test MSE vs* `n_train_post` plots:

- At iteration 0 the blue curve (current weights) lies exactly on top of the orange curve (all weights = 1), and both are clearly above the green oracle curve.
- As meta-training proceeds, the blue curve moves steadily downward and gets very close to the green oracle curve, especially in the low–to–moderate data regime. It is consistently below the orange curve, meaning the meta-learned feature weights give lower test MSE than using all ones, although they still do not perfectly match the oracle. For very large `n_train_post` all curves nearly coincide, since with enough data the choice of feature weights matters less.

The sharp downward spike at `n_train_post = 32` occurs because the inner MAML training always uses `n_train_inner = 32` examples. The meta-objective is therefore optimized specifically for "32-point" least-squares fits on this grid, so the learned feature weights are best adapted to that sample size. When we evaluate with exactly 32 post-training points, the evaluation distribution matches the training distribution, and the test loss dips sharply. For other `n_train_post`, we are slightly off the distribution the meta-learner was tuned for, so the loss is higher and the spike disappears.

(e)

Looking at the *feature-weight* plots over iterations:

- Initially all 501 feature weights are equal to 1, so the model treats every Fourier feature as equally useful.
- After meta-training, only a small band of features around the true indices `k_idx = 5,…,10` retain large magnitudes, while almost all other feature weights are pushed close to zero.

This pattern explains the performance gain. Across tasks, the target functions are always built from the `k_idx` features; all other features behave like redundant or noisy basis functions. The meta-loss therefore pushes weights on consistently useful features (those in `k_idx`) up, and on unhelpful or aliased features down. Driving those irrelevant weights toward zero effectively reduces the model's capacity and shrinks the min-norm solution onto the true subspace, which lowers variance and improves test MSE—hence the blue curve moves toward the oracle as training progresses.

(f)

From the *regression with GD* figures:

- **When** `num_gd_steps = 5`
  - At iteration 0, the blue curve ("Iteration 0") in *Test mse vs* `n_train_post` lies almost exactly on top of the orange curve ("Init"), well above the green oracle curve.
  - After meta-training (later iterations), the blue curve moves clearly **below** the orange curve across almost all values of `n_train_post`, and for small–to–moderate `n_train_post` it sits quite close to the green oracle curve.
  - This means that with 5 inner GD steps, the meta-learned feature weights do give **lower test MSE than the untrained all-ones weights**, and they nearly recover the oracle behaviour in the data-poor regime. So meta-learning *does* improve test-time performance.
- **When** `num_gd_steps = 1`
  - A similar pattern appears: starting from iteration 0, blue ≈ orange and both are far from the oracle; as meta-training proceeds, the blue curve again drops below the orange curve for most `n_train_post`
  - However, the gap between blue and green is larger than in the 5-step case: the blue curve no longer tracks the oracle as closely, especially for intermediate `n_train_post`.
  - So with only a single inner GD step, **meta-learning still works**—the learned feature weights give better test MSE than the all-ones baseline—but the improvement is weaker, because the inner optimization is much more approximate and the representation has to compensate for doing only one update.

(g)

Looking at the *test classification error vs* `n_train_post` plots:

- At **iteration 0**, the blue curve (current weights) sits almost exactly on top of the **orange** curve (all feature weights = 1) and is well above the **green** oracle curve. So before meta-learning, using meta-parameters is no better than the naive all-ones baseline, and both are worse than the oracle that knows exactly which features matter.
- After meta-training, the **blue curve moves consistently below the orange curve** across almost the entire range of `n_train_post`. This means the meta-learned feature weights give **lower classification error** than using all ones, especially in the low-data regime (small `n_train_post`), which is exactly what MAML is supposed to do: it reduces the amount of data needed per task by learning a good initialization / representation.
- The blue curve also moves **closer to the green oracle curve**. It typically does not match the oracle perfectly, but it often tracks the oracle fairly closely for moderate `n_train_post`. So meta-learning narrows the gap toward the oracle classifier that is allowed to use only the true task features.

In short: meta-learned weights clearly outperform the all-ones baseline and partially close the gap to the oracle, especially when only a limited number of labeled points are available for each new task.

(h)

The *feature-weight* plots (right panels of the second row) show why this happens:

- At **iteration 0**, all 501 feature weights are exactly 1, so the logistic regression in the inner loop treats every Fourier feature equally.
- After meta-training, only a small group of features—corresponding to the true indices in `k_idx`—retain large magnitude. **Most other feature weights are driven close to zero.**

Intuitively, across tasks the labels are always generated from the same subset of "true" features, while the remaining features behave like irrelevant or noisy directions. The meta-objective averages classification loss across many tasks, so:

- Features that *consistently help* explain the labels receive gradients that keep their weights large.
- Features that *never systematically help* just add variance and overfitting; their contributions hurt expected task performance, so the meta-gradients push their weights toward zero.

By shrinking these unhelpful feature weights, meta-learning effectively performs **task-aware feature selection / regularization**: the inner logistic regression sees a lower-dimensional, better-aligned representation, which yields cleaner decision boundaries from few examples and hence lower test error. This matches the observed improvement of the blue curve over the all-ones baseline and its movement toward oracle performance.

---

6. Homework Process and Study Group

(a)

Lecture Notes

(b)

None

(c)

1 day